

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Задание № 1

По дисциплине

“Алгоритмы компьютерной графики”

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель: Андреев Артем Станиславович

Санкт-Петербург, 2025г

Задание

Этот пример в DirectX9 директории. В 10 и 11 его подменили простым примером.
Взяв любые примеры – OpenGL GLSL, HLSL пример отрисовки, преобразовать вывод звезды с 5 лучами на иное число лучей – 4, 6.
Поменять цвет фона, заданный в исполняемом файле.
Реализовать вместо triangle fan : triangle list и triangle strip.

Ход работы

1. Исходный код

Полный модифицированный исходный код примера:

```
#include <d3d9.h>
#pragma warning( disable : 4996 ) // disable deprecated warning
#include <strsafe.h>
#include <cmath>
#pragma warning( default : 4996 )

//-----
// Global variables
//-----

LPDIRECT3D9          g_pD3D = NULL; // Used to create the D3DDevice
LPDIRECT3DDEVICE9    g_pd3dDevice = NULL; // Our rendering device
LPDIRECT3DVERTEXBUFFER9 g_pVB = NULL; // Buffer to hold vertices

// A structure for our custom vertex type
struct CUSTOMVERTEX
{
    FLOAT x, y, z, rhw; // The transformed position for the vertex
    DWORD color;         // The vertex color
};

// Our custom FVF, which describes our custom vertex structure
#define D3DFVF_CUSTOMVERTEX (D3DFVF_XYZRHW|D3DFVF_DIFFUSE)

//-----
// Name: InitD3D()
// Desc: Initializes Direct3D
//-----

HRESULT InitD3D( HWND hWnd )
{
    // Create the D3D object.
    if( NULL == ( g_pD3D = Direct3DCreate9( D3D_SDK_VERSION ) ) )
        return E_FAIL;
    // Set up the structure used to create the D3DDevice
    D3DPRESENT_PARAMETERS d3dpp;
    ZeroMemory( &d3dpp, sizeof( d3dpp ) );
    d3dpp.Windowed = TRUE;
    d3dpp.SwapEffect = D3DSWAPEFFECT_DISCARD;
```

```

d3dpp.BackBufferFormat = D3DFMT_UNKNOWN;

// Create the D3DDevice
if( FAILED( g_pd3D->CreateDevice( D3DADAPTER_DEFAULT, D3DDEVTYPE_HAL,
hWnd,

D3DCREATE_SOFTWARE_VERTEXPROCESSING,

&d3dpp, &g_pd3dDevice ) ) )
{
    return E_FAIL;
}

// Device state would normally be set here
g_pd3dDevice->SetRenderState(D3DRS_CULLMODE, D3DCULL_NONE);
return S_OK;
}

// Задаем число лучей звезды
int NUM_POINTS = 5;
LPDIRECT3DINDEXBUFFER9 g_pIB = NULL; // Глобальная переменная для буфера
индексов

HRESULT InitIB() {
    int numPoints = NUM_POINTS;
    int numTriangles = numPoints; // Звезде нужно N треугольников
    WORD* indices = new WORD[3 * numTriangles];
    int idx = 0;

    for (int i = 0; i < 2 * numPoints; i += 2) {
        int nextOuter = (i + 2) % (2 * numPoints);
        // ОДИН треугольник на луч: (внешняя_точка[i],
внутренняя_точка[i], внешняя_точка[i+2])
        indices[idx++] = i;           // Внешняя точка i
        indices[idx++] = i + 1;       // Внутренняя точка i
        indices[idx++] = nextOuter;   // Следующая внешняя точка (через
одну)
    }

    // Создание буфера индексов
    if (FAILED(g_pd3dDevice->CreateIndexBuffer(3 * numTriangles *
sizeof(WORD),
        0, D3DFMT_INDEX16, D3DPOOL_DEFAULT, &g_pIB, NULL))) {
        delete[] indices;
        return E_FAIL;
    }

    // Копирование данных
    VOID* pIndices;
    if (FAILED(g_pIB->Lock(0, 3 * numTriangles * sizeof(WORD),
(void**)&pIndices, 0))) {
        delete[] indices;
        return E_FAIL;
    }
    memcpy(pIndices, indices, 3 * numTriangles * sizeof(WORD));
    g_pIB->Unlock();
    delete[] indices;
    return S_OK;
}

```

```
//-----
-----
// Name: InitVB()
// Desc: Creates a vertex buffer and fills it with our vertices. The
vertex
//      buffer is basically just a chunk of memory that holds vertices.
After
//      creating it, we must Lock()/Unlock() it to fill it. For indices,
D3D
//      also uses index buffers. The special thing about vertex and
index
//      buffers is that they can be created in device memory, allowing
some
//      cards to process them in hardware, resulting in a dramatic
//      performance gain.
//-----
-----
HRESULT InitVB()
{
    int numPoints = NUM_POINTS;          // Количество лучей
    float R_out = 100.0f;                 // Внешний радиус
    float R_in = 40.0f;                   // Внутренний радиус
    int totalVertices = 2 * numPoints;    // Звезда без центральной точки
для TriangleList
    CUSTOMVERTEX* vertices = new CUSTOMVERTEX[totalVertices];

    float angleStep = 2.0f * 3.14159f / (2 * numPoints);
    for (int i = 0; i < 2 * numPoints; ++i)
    {
        float angle = i * angleStep;
        float radius = (i % 2 == 0) ? R_out : R_in;
        vertices[i].x = 150.0f + radius * cosf(angle);
        vertices[i].y = 150.0f + radius * sinf(angle);
        vertices[i].z = 0.5f;
        vertices[i].rhw = 1.0f;
        vertices[i].color = 0xffffffff;
    }

    // Создание вершинного буфера
    if (FAILED(g_pd3dDevice->CreateVertexBuffer(totalVertices *
sizeof(CUSTOMVERTEX),
        0, D3DFVF_CUSTOMVERTEX, D3DPOOL_DEFAULT, &g_pVB, NULL)))
    {
        delete[] vertices;
        return E_FAIL;
    }

    // Копирование данных в буфер
    VOID* pVertices;
    if (FAILED(g_pVB->Lock(0, totalVertices * sizeof(CUSTOMVERTEX),
(void**) &pVertices, 0)))
    {
        delete[] vertices;
        return E_FAIL;
    }
    memcpy(pVertices, vertices, totalVertices * sizeof(CUSTOMVERTEX));
    g_pVB->Unlock();
}
```

```

        delete[] vertices;

        return S_OK;
    }

//-----
// Name: Cleanup()
// Desc: Releases all previously initialized objects
//-----
VOID Cleanup()
{
    if (g_pIB != NULL)
        g_pIB->Release(); // Освобождаем индексный буфер

    if (g_pVB != NULL)
        g_pVB->Release(); // Освобождаем вершинный буфер

    if (g_pd3dDevice != NULL)
        g_pd3dDevice->Release();

    if (g_pD3D != NULL)
        g_pD3D->Release();
}

//-----
// Name: Render()
// Desc: Draws the scene
//-----
VOID Render()
{
    // Задаем цвет фона
    g_pd3dDevice->Clear(0, NULL, D3DCLEAR_TARGET, D3DCOLOR_XRGB(255, 0,
0), 1.0f, 0);
    if( SUCCEEDED( g_pd3dDevice->BeginScene() ) )
    {
        g_pd3dDevice->SetStreamSource( 0, g_pVB, 0,
sizeof( CUSTOMVERTEX ) );
        g_pd3dDevice->SetFVF( D3DFVF_CUSTOMVERTEX );
        g_pd3dDevice->SetIndices(g_pIB);
        g_pd3dDevice->DrawIndexedPrimitive(D3DPT_TRIANGLELIST, 0, 0, 0,
0, NUM_POINTS);
        // End the scene
        g_pd3dDevice->EndScene();
    }

    // Present the backbuffer contents to the display
    g_pd3dDevice->Present( NULL, NULL, NULL, NULL );
}

//-----

```

```

// Name: MsgProc()
// Desc: The window's message handler
//-----
-----
LRESULT WINAPI MsgProc( HWND hWnd, UINT msg, WPARAM wParam, LPARAM
lParam )
{
    switch( msg )
    {
        case WM_DESTROY:
            Cleanup();
            PostQuitMessage( 0 );
            return 0;
    }

    return DefWindowProc( hWnd, msg, wParam, lParam );
}

//-----
-----
// Name: wWinMain()
// Desc: The application's entry point
//-----
-----
INT WINAPI wWinMain( HINSTANCE hInst, HINSTANCE, LPWSTR, INT )
{
    UNREFERENCED_PARAMETER( hInst );
    WNDCLASSEX wc =
    {
        sizeof( WNDCLASSEX ), CS_CLASSDC, MsgProc, 0L, 0L,
        GetModuleHandle( NULL ), NULL, NULL, NULL, NULL,
        L"D3D Tutorial", NULL
    };
    RegisterClassEx( &wc );
    HWND hWnd = CreateWindow( L"D3D Tutorial", L"D3D Tutorial",
                             WS_OVERLAPPEDWINDOW, 100, 100, 300, 300,
                             NULL, NULL, wc.hInstance, NULL );

    // Initialize Direct3D
    if( SUCCEEDED( InitD3D( hWnd ) ) )
    {
        // Create the vertex buffer
        if( SUCCEEDED( InitVB() ) )
        {
            if (SUCCEEDED(InitIB()))
            {
                // Show the window
                ShowWindow(hWnd, SW_SHOWDEFAULT);
                UpdateWindow(hWnd);

                // Enter the message loop
                MSG msg;
                ZeroMemory(&msg, sizeof(msg));
                while (msg.message != WM_QUIT)
                {
                    if (PeekMessage(&msg, NULL, 0U, 0U, PM_REMOVE))
                    {
                        TranslateMessage(&msg);
                        DispatchMessage(&msg);
                    }
                }
            }
        }
    }
}

```

```

        }
        else
            Render();
    }
}

}

UnregisterClass( L"D3D Tutorial", wc.hInstance );
return 0;
}

```

2. Комментарии к выполнению

Редактирование и запуск исходного кода осуществлялось в Visual Studio.
 Была изменена отрисовка звезды с использованием triangle list для отрисовки.

Для изменения цвета фона нужно внести изменения в эту строку кода:

```

// Задаем цвет фона
g_pd3dDevice->Clear(0, NULL, D3DCLEAR_TARGET, D3DCOLOR_XRGB(255, 0, 0), 1.0f, 0);

```

В параметре D3DCOLOR_XRGB(255, 0, 0) указываем нужный нам цвет, я указал красный.
 Для того чтобы использовать triangle list для отрисовки я реализовал функцию HRESULT InitIB()

Также я вынес в глобальную переменную значение числа вершин, благодаря чему можно легко изменять число отрисовываемых вершин звезды.

Вот строка кода для изменения числа вершин:

```

// Задаем число лучей звезды
int NUM_POINTS = 5;

```

3. Примеры отрисовок

Для отрисовки звезд с разным количеством вершин, просто будем изменять значение глобальной переменной int NUM_POINTS

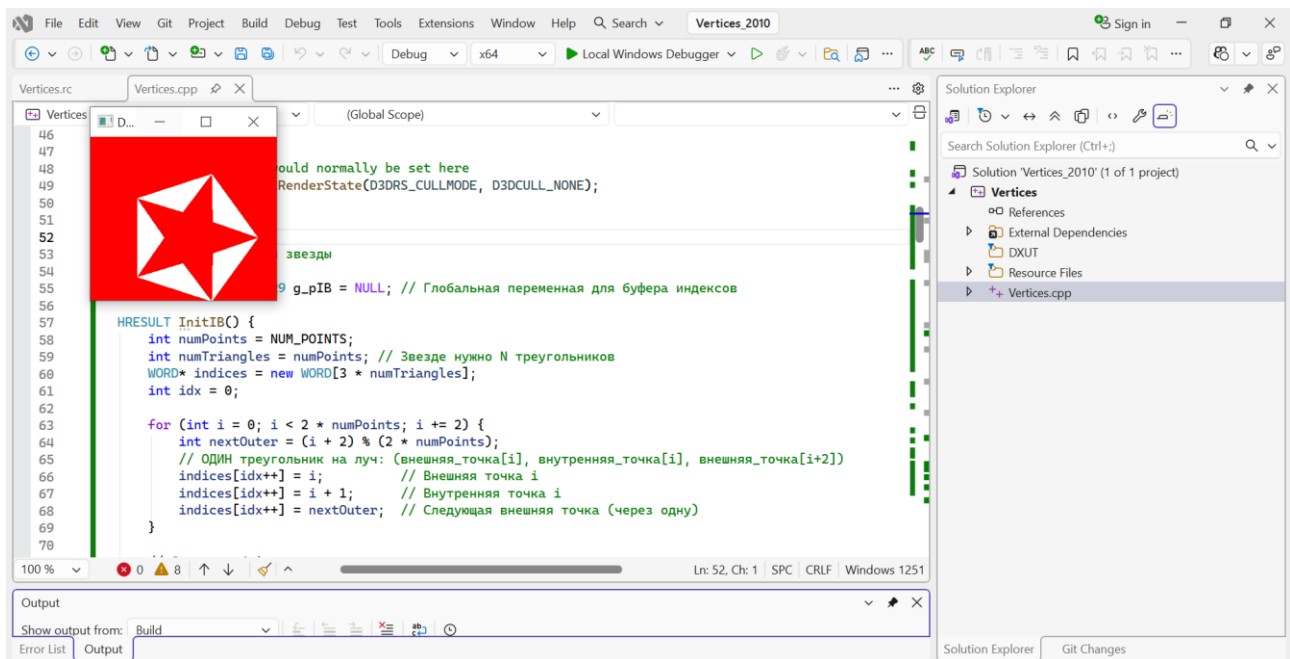


Рисунок 1. Отрисовка звезды с 5 лучами

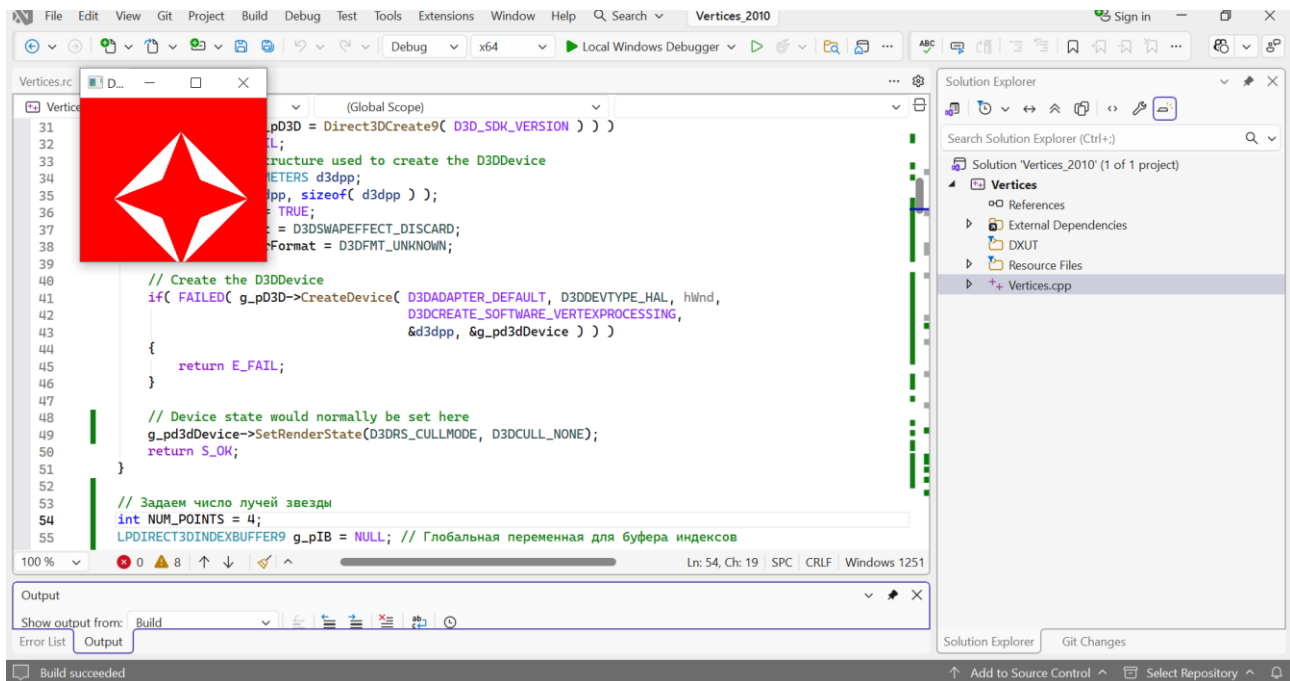


Рисунок 2. Отрисовка звезды с 4 лучами

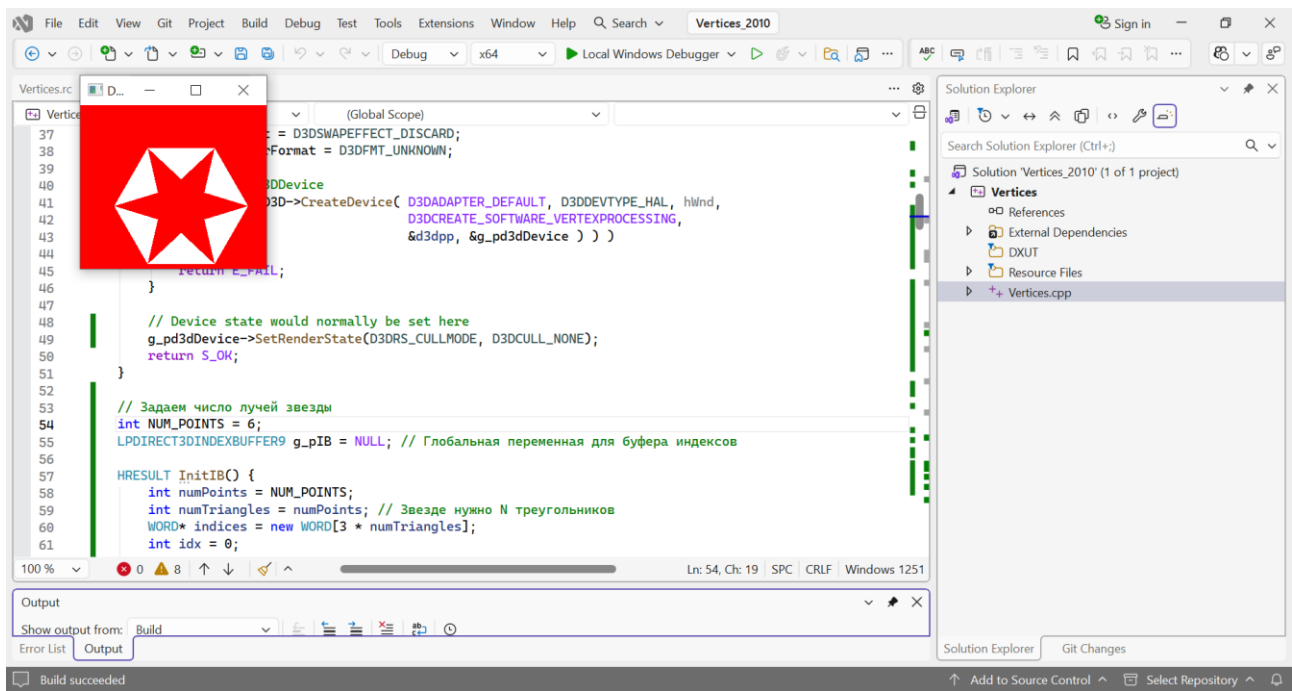


Рисунок 3. Отрисовка звезды с 6 лучами

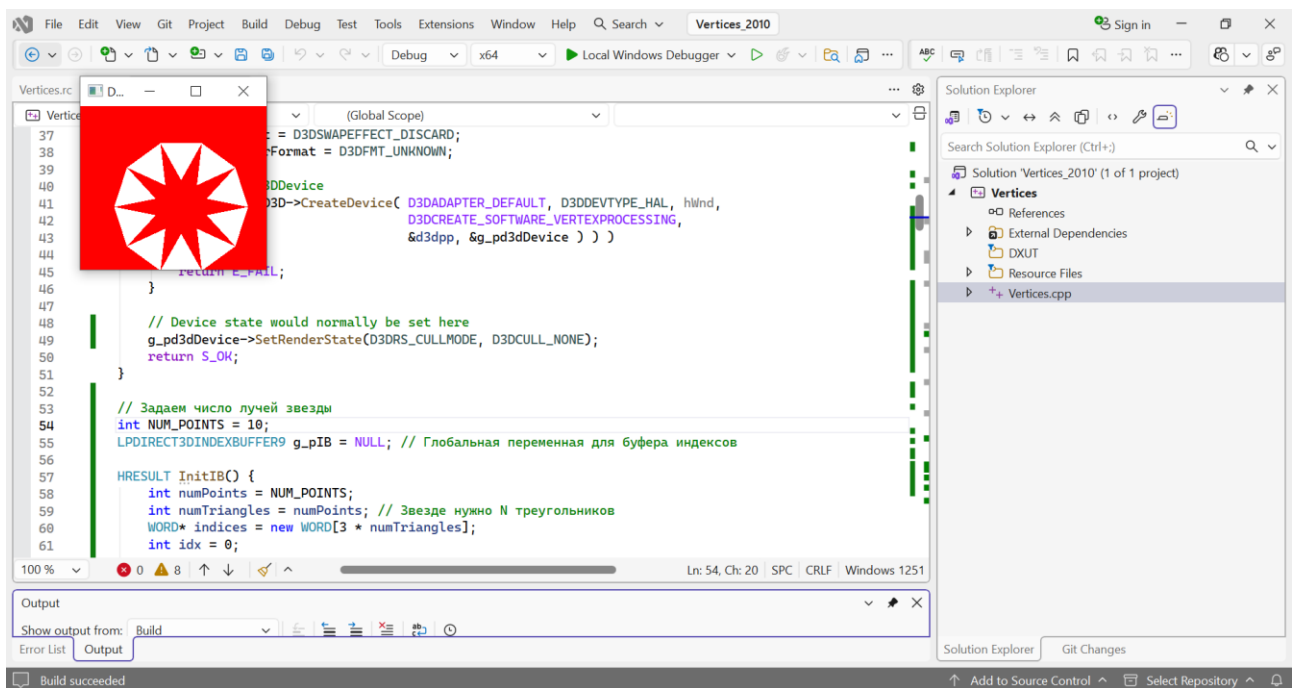


Рисунок 4. Отрисовка звезды с 10 лучами